

BPsec-MLS: Asynchronous Key Agreement for Space Communications

Xisen Tian
Naval Postgraduate School
xisen.tian1@nps.edu

Paul Westland
Naval Postgraduate School
paul.westland@nps.edu

Abstract—Key agreement is the cornerstone of any secure communications channel whether over the traditional internet or in delay tolerant networks used in space communications. However, space systems that rely on pre-shared keys face insurmountable limitations in both security and scalability. A single key compromise can expose all past and future encrypted communications, and the static nature of pre-shared keys prevents dynamic group membership, making it difficult to add or remove devices without invalidating entire key sets. To address these challenges, the Messaging Layer Security (MLS) protocol – a recently introduced internet standard for asynchronous group key establishment – offers promising capabilities. Its potential to provide efficient and dynamic key agreement for federated interplanetary architectures (e.g. government-commercial collaborations) has been previously recognized, particularly with integration of MLS with the Bundle Protocol Security (BPsec) framework. In this work, we present the first empirical findings on the integration of MLS with BPsec in a realistic space communications scenario and provide insights into its deployment in future space architectures.

I. INTRODUCTION

The growing domain of space scenarios [1], [2], [3] requiring interoperability, efficient scaling, and strong security guarantees is outpacing the limits imposed through use of pre-shared keys and synchronous handshake protocols [4], [5], [6]. Pre-shared symmetric keys limit networks from being able to dynamically add or remove users. Moreover, compromise of a pre-shared key reveals all past and future communications to an adversary. Alternatively, synchronous handshake protocols allow for dynamic session setups which protects previous communications from key compromise, however, their interactive nature is unsuited for delay and disruption prone networks [7].

In alignment to the requirements of delay tolerant networks (DTN), the Messaging Layer Security (MLS) protocol [8] was designed for scalable asynchronous group key agreement. MLS achieves forward secrecy (FS) and post-compromise security (PCS) to protect previous and future communications against key compromise through the evolution of a shared group state across epochs in a continuous session. FS guarantees that past conversations remain secure after a compromise of the current session key. PCS guarantees that future communications are secure after a compromise.

The ability to efficiently and continuously update keys and dynamically add or remove group members *post-launch* is a

particular advantage of using MLS that is currently absent from the Bundle Protocol Security (BPsec) which used to provide application data confidentiality and integrity in DTNs. In terms of efficiency, MLS can update group keys with communication costs as low as logarithmic in the best case, though in practice the cost is usually closer to linear on average [9], [10], [11]. By contrast, synchronous protocols that rely on fresh Diffie-Hellman values (i.e. starting a new session to achieve comparable FS/PCS), incur polynomial overhead in relation to the group size. If keys are updated by bootstrapping off the existing session keys then FS and PCS are lost.

Dowling et al. in [12] first outlined the merits of MLS as a key agreement protocol for space communications architectures, highlighting many of the aforementioned issues. This work implements their proposal to incorporate MLS in BPsec and maps it to relevant space scenarios and measures costs of such an incorporation. Our non-intrusive approach ensures that the reference libraries of BPsec and MLS do not deviate from their standards, thus reducing systems engineering barriers to deploying BPsec-MLS.

We make the following contributions in this work:

- To the best of our knowledge, we provide the first implementation of a continuous group key agreement protocol for delay tolerant networks. Furthermore, our implementation is in compliance with both BPsec and MLS standards and is tailored for space scenarios through incorporation of MLS in BPsec.
- We detail practical considerations for incorporating MLS architectural components to the DTN environment.
- We show MLS overhead in a full DTN stack covering two group administration strategies and present their trade-offs in terms of efficiency and security guarantees.

II. BACKGROUND

We provide preliminaries and relevant context to key agreement protocols, delay tolerant networks, and efforts to combine the two in the following section.

A. Continuous Group Key Agreement Protocols

The Signal protocol's double ratchet algorithm inspired formalization of the concept of a Continuous Key Agreement (CKA) protocol by Alwen et al. in [13] as a cryptographic primitive that enables two parties to agree on a shared secret

key that evolves over time. As an extension of CKA, Continuous Group Key Agreement (CGKA) is used in dynamic multiparty settings where members may join, leave, or update their keys frequently [14]. CGKA protocols provide strong security properties such as forward secrecy, post-compromise security, and efficient group operations. TreeKEM is a specific CGKA protocol that uses a tree-based structure to manage keys and efficiently handle group changes. It serves as the cryptographic backbone of MLS and has been formally analyzed and improved upon in lockstep [15], [16], [17].

Messaging Layer Security Protocol: The MLS protocol was standardized as RFC9420 in 2023 by the Internet Engineering Task Force (IETF) to provide end-to-end security [8]. As an open and interoperable standard, it has been deployed commercially by Cisco Webex, RingCentral, and Wire [18]. MLS uses the TreeKEM protocol to maintain a shared cryptographic state upon which keys for various purposes (e.g., key encapsulation, encryption, signing, etc.) may be derived. The MLS architecture document describes various components necessary to run MLS [19]. To add a new group member, the sender first acquires a KeyPackage associated to the invitee signed by an authentication service. Then, using the KeyPackage, the sender adds the member to their local group state generates a Welcome message for the invitee to allow them to calculate the group state and join the group. Because this changes the group state, the sender must also send an Add proposal which must be committed (either through another member or via self-commit) to synchronize the new group state. To achieve PCS, members should also issue an updates that are committed periodically.

In general, MLS group members advance the shared cryptographic state (via adding, removing, updating) through a propose-and-commit scheme. This characteristic, which maximizes autonomy of members, can lead to synchronization issues if the underlying delivery service is unreliable [19]. For this reason, our DTN implementation of MLS which may be prone to disruptions and delays delegates the ability to commit update proposals to a group leader. Naively applied, this paradigm can create a ratchet tree where a majority of the intermediate nodes are blanked which nullifies any logarithmic scaling provided by a well populated ratchet tree. However, this may be avoided if each even indexed member is allowed to issue an update and commit the first time they join as part of the group instantiation process. We elaborate further on this in section III-C.

Performance Analysis: Theoretical asymptotic analysis of CGKAs has been previously analyzed by Beinstock, et al. in [20] who show CGKAs generally achieve $\Omega(n)$ worst-case communication complexity. Anastos, et al. in [21] refine this by showing the cost (number of uploaded ciphertexts) of replacing a set d users in a group of size n is $\Omega(d \ln(n/d))$. Theoretical and experimental performance analysis of Server Aided Insider-Secure TreeKEM, was performed by Alwen, et al. in [10]. They show, at the scale of tens of thousands of members in the client-server setting, communications bandwidth in best-cases can scale logarithmically for individual

senders and receivers; however, in the worst-case scenarios bandwidth scaling was still linear for senders (servers) the total combined bandwidth of the group was $\Omega(n \log(n))$. Chavelier, et. al in [22] provide further insight on the variability of communications performance depending on the configuration of the MLS ratchet tree. They highlight inefficiencies associated with the MLS left-balanced ratchet tree expansion algorithm and note that the lower bound logarithmic scaling would only be attainable under the ideal ratchet tree structure (i.e. a full balanced binary tree) which can only occur if the group size is a power of two. Soler, et al. in [11] provide another experimental analysis of MLS using different delivery services. Soler, et al. show under practical configurations and various group sizes ranging to one thousand, that the computational and storage costs scale generally linear rather than logarithmic as has been previously claimed.

B. Delay Tolerant Networks (DTN)

DTNs describe a collection of protocols developed by NASA originally for deep space communication that follow a store-and-forward paradigm. DTN protocols exist for each level of the network stack. As a DTN counterpart to IP, the Bundle Protocol (BP) [23] when used with BPsec [24], is a secure DTN application messaging protocol with integrated transport functionalities. BPsec uses Block Integrity Blocks (BIB) and Block Confidentiality Blocks (BCB) to provide data authentication and confidentiality to blocks within a bundle according to local security policies. BIB and BCB are cryptographically parameterized by a Security Context, configured and loaded out-of-band. Formal analysis of BPsec under the Default Security Context has been analyzed in [25].

Unlike other secure transport protocols such as QUIC, BPsec explicitly does not provide the function of authenticated key exchange [26]. Instead, users must perform entity authentication and key agreement out-of-band (e.g. via pre-installed keys). In this regard, BPsec falls short of providing users with modern end-to-end security guarantees. One attempt to bridge this gap has been explored in [27] on a QUIC convergence layer protocol (QUICL) for BP which uses Transport Layer Security (TLS) for key agreement. However, as [12] argues, several issues remain with using standard QUIC with TLS in high-latency and intermittently connected networks. In a recent draft [28] submitted to the IETF, the authors propose to use QUIC with MLS instead of TLS for use in IP based space communications settings. However, until a convergence layer adapter is created for QUIC-MLS, it won't be compatible with BP networks.

Non-standardized Approaches to Key Agreement: Various efforts have tried to incorporate key agreement in delay tolerant networks and space scenarios. Recently, the TripleKEM algorithm was proposed for the Space Data Link Security (SDLS) protocol [29]. However, the interactive nature of TripleKEM limits its scalability to go beyond hop-to-hop pairwise scenarios endemic to the link layer. Another drawback of TripleKEM and other key agreement protocols proposed for DTNs [30], [31], [32] is that none have been standardized,

made open-source, nor are widely adopted. On the other hand, MLS is an open-source, standardized protocol, commercially deployed, and has been thoroughly tested and vetted among academic and commercial researchers for its security, performance, and interoperability [17], [33], [16], [34].

III. DESIGN

In this section we explore the design space associated with implementing MLS as a key agreement mechanism in BPsec. End-to-end security (E2E) and efficiency guarantees aforementioned differ in degree depending on how MLS is integrated and on assumptions of MLS architectural components.

A. Integration Rationale

MLS provides E2E authenticated key agreement between group members. In BP networks, trusted (aka *privileged*) intermediate nodes between a source and destination with a shared key-set must sometimes add or process BPsec blocks (e.g. performance enhancement proxies). This inherent flexibility of BPsec which facilitates the store-and-forward paradigm is at odds with the E2E paradigm in the strictest sense of the meaning [35]: that only endpoints should handle the security functionality.

If a strict E2E secure channel is required, then a simple approach might be to use MLS *on top of* BP. This would include sending MLS protected application messages in addition to MLS handshake messages in the payload (see right side of Figure 1). This design relegates intermediate nodes to only function as forwarders, with no insight on the encrypted payload, which may be incompatible with networks that use proxies or gateways for security services. This approach would also subvert the flexibility of BPsec in providing block-level granularity and interactions among security operations as analyzed in [25]. In fact, MLS would obviate BPsec entirely as this tunneling technique includes both authenticated key agreement and confidential authenticated messaging.

In order to preserve the flexibility of BPsec while providing the option to enforce E2E design (i.e., by excluding intermediate nodes), our implementation integrates MLS into BPsec while maintaining compliance to the current standards [24], [26]. Our design, shown in Figure 1, sends MLS handshake messages through the payload block. Alternatively, a more efficient design would have MLS messages be carried in a designated extension block type or administrative record [36] but we chose to not do so to maintain compliance with current standards. Once received, bundles containing MLS messages are processed by recipients to establish or update a shared cryptographic state from which keys are derived in accordance with the MLS Key Schedule [8]. Cryptographic parameters and capabilities are also conveyed through the group state which can be checked against security contexts supported by group members. Rather than implementing this feature, we hard-coded the parameters to match.

B. Architectural Components and Assumptions

1) *Delivery Service*: MLS group operations are enacted through a propose-and-commit sequence which necessitates

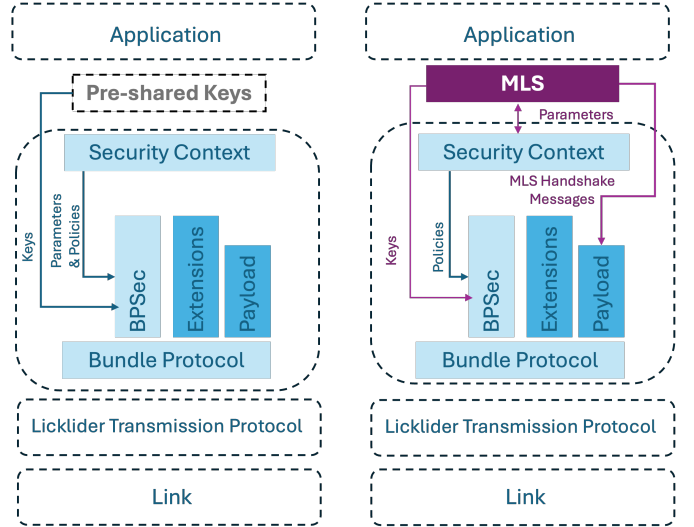


Fig. 1: BPsec using out-of-band (pre-shared) keys (left) versus our BPsec in-band key agreement using MLS (right)

a Delivery Service (DS) that ensures the ordered delivery of these messages and resolves any conflicts. When a MLS group member sends a group operation proposal (e.g. add, remove, update), any member of the MLS group may send a commit to the list of proposals and move the group state forward. The DS is an MLS architectural assumption that decides the ordering and precedence of proposals and commits which helps to keep the common group state synchronized. In existing implementations on the internet, the DS role is relegated to either the underlying transport protocol such as TCP or a centralized server.

After consideration of alternatives (see Section A), our DS is fulfilled by using the Licklider Transport Protocol (LTP) for reliable delivery of messages and having a single group leader to reconcile proposals and sending commits [37].

2) *Authentication Service*: The role of the Authentication Service (AS) for MLS is to associate an identity to a signature key pair [19]. In the DTN setting, several systems have been proposed that can fulfill the role of the AS. For example, as a public key distribution service, DTKA can fulfill the role of the AS. More recently, authors in [5] demonstrated the viability of standard terrestrial style public key infrastructure through customization of the Online Status Certificate Protocol and Certificate Revocation List delivery methods based on the same set of stipulations set by DTKA in [38]. Since a litany of AS candidates exist, we leave AS design analysis and comparison for future work and simply preset identity bindings to signature keys for our implementation.

C. Network Scenario Alignment

Our implementation uses a hub-spoke topology as shown in Figure 2 for key agreement that is aligned with standard space architectures and scenarios described by the Consultative Committee for Space Data Systems in [39]. Specifically, we

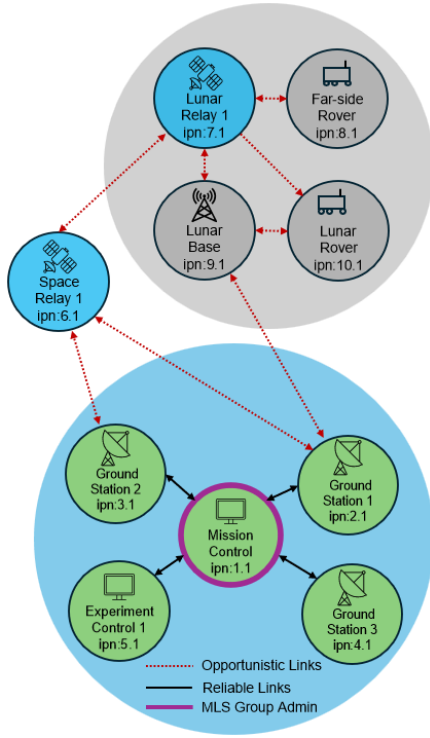


Fig. 2: Hub-spoke MLS key agreement overlayed on DTN scenario adapted from [39]. Mission Control (purple) acts as the leader responsible for fanning out MLS commits to the rest of the members with varying connections.

model communications between terrestrial control center(s) to lunar terminals and relay orbiter(s) to lunar terminals.

The well-connected hub is the MLS *leader* group member that adjudicates commitments and proposals sent by variably connected *subordinates* group members. Subordinates, which may or may not be well connected among each other, can then use the group key to communicate directly within any topology using simplex or duplex channels.

The choice to consolidate commit power to the leader reduces the complexity in managing group state through unilateral arbitration of proposals for group consensus. It also reduces the computation and bandwidth burden on subordinates. However, this leader-subordinate paradigm can come at a cost to MLS performance and security guarantees depending on how the group is administered. We present three configuration options for consideration:

- 1) *Leader Only*: If subordinates never update themselves and only apply commits from the leader, then only the leader can achieve PCS and subordinates gain FS only. Furthermore, the MLS ratchet tree ostensibly flattens to a linked list which negates the logarithmic scaling benefits of a well structured tree [22].
- 2) *Subordinate Proposal*: If subordinates are allowed to propose updates but not ever commit, then they would regain PCS but still retain the poorly formed ratchet tree with blank intermediate nodes along the direct paths of subor-

dinate nodes. Subordinates have more agency in updating but commit sequencing and conflict resolution still rest with the leader.

- 3) *Subordinate Commit*: If subordinates are allowed to propose updates *and* commit their own update (e.g. just after joining the group), then both FS and PCS can be attained AND the ratchet tree could be filled and balanced so logarithmic efficiencies could be realized. The leader acts as a commit forwarder here and additional out-of-band coordination would be required to ensure no conflicting commits arrive.

Of the three leader-subordinate configurations, Option 1 is the most accommodating to DTN scenarios where subordinates can be severely resource constrained (compute, bandwidth, power, etc.) and delegation of MLS operations to the leader should be maximized. Options 2 and 3 achieve better security outcomes but requires more subordinate resources and potentially more coordination (i.e. timers, event logging, conflict resolution, etc) to ensure group state synchronization. For testing and evaluation purposes, since Option 2 maintains a poor ratchet like in Option 1, our implementation examines only Options 1 and 3 (i.e. worst-case and best-case settings). Summarily, we hypothesize Option 3 to have lower overhead than Option 1. In practice, we recommend a combination of Option 2 and 3 to achieve the best security and efficiency outcomes. That is, subordinates should commit their own update after being added as part of group instantiation and subsequently only send proposals to the leader for commitment to ameliorate the concerns of out-of-order or conflicting commits due to delay or packet loss.

D. Parameter Selection

Relevant to using MLS for space based CGKA are computation cost, memory use, and size of the data transmitted. These metrics directly correspond to the practical constraints faced by onboard spacecraft computers and resource-constrained embedded systems. Compute time was measured to assess the processing burden imposed by cryptographic operations, particularly for handling welcome and update messages, since onboard CPUs often operate at lower clock speeds and under strict power budgets. Monitoring CPU usage allows us to attribute operations that could delay critical tasks or exceed real-time processing windows. Memory usage was tracked to ensure that the MLS session state and buffer handling remained within the limited memory typically available on flight-qualified hardware, where excessive heap allocation could cause failures to mission critical programs. Packet size was also recorded, as spectrum availability and bandwidth are both severely limited in deep space environments. Larger bundles result in longer transmission times and higher energy costs per bit transmitted. By collecting these metrics, we provide tradeoffs between cryptographic security guarantees and the limitations of a device's physical hardware and communication channels in which these systems must operate.

IV. IMPLEMENTATION SETUP

Our setup ran inside a virtual machine (VM) on a personal computer with 64GB of RAM and an AMD Ryzen 5 3600 six-core processor. The Linux-based VM was configured with two cores and 54 GB of RAM. We used the Interplanetary Overlay Network (ION) software suite [40], which implements the Bundle Protocol [23], BPsec [41], and the Default Security Context [26]. For MLS, our MLS Daemon utilizes Cisco’s MLSp library [42].

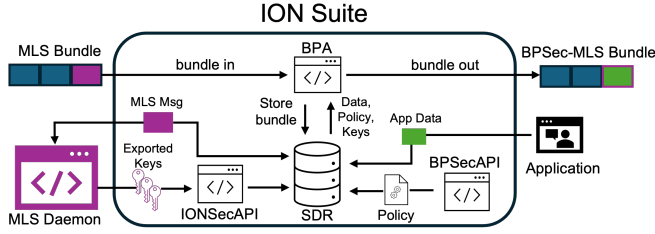


Fig. 3: Simplified overview of components of an ION node running BPsec with MLS. Implementation details available at <https://github.com/xisentian/ion-dtn-mls>.

Each ION node is configured in accordance with fig. 3 and has its own MLS Daemon. The Bundle Protocol Agent (BPA) handles bundle transmission and reception. The Simple Data Recorder (SDR) stores data, policies, and keys used in BP operations. The ION Security API is used to read and write MLS keys to the SDR. The BPsec API is used to load a security policy into the SDR, which is referenced by the BPA when bundles are received or transmitted. If the policy specifies that security operations such as encryption or decryption are required, the policy also specifies the key to be used for those operations. Our MLS Daemon handles MLS messages parsed by the BPA for group creation, adding and removing members, as well as updating the group state. Depending on the role of the ION node (leader or subordinate), which is assigned by policy (out-of-band to MLS), the MLS Daemon allows the particular node to issue commits to proposals sent by other group members.

MLS group instantiation occurs with the leader adding all subordinates en masse which results in the creation of a single commit message (referencing a list of add proposals) and welcome message, both created from subordinate key packages generated using their unique endpoint IDs. After a new member processes the Welcome message contained in a bundle payload, the MLS daemon constructs the group state and is able to immediately process (e.g., encrypt, decrypt, sign, verify) BCB or BIB protected data. This concludes group instantiation for the configuration described in section III-C for Option 1. For Option 3, an additional step is required to populate the contents of parent nodes in the MLS ratchet tree: every even-indexed member sends an empty commit to the leader to be broadcast after being added to the group. Once the MLS group has been established, keys can be updated via

commits as frequently as needed. Ideally, all members should update on a regular basis to attain PCS.

V. RESULTS

Following the setup and group evolution sequences (Options 1 and 3) described in section IV, we measured the computational and bandwidth costs of joining a group and processing an update shown in fig. 4. Since both options share the step of processing a Welcome message when subordinates join a group, only one set of data is shown. Measurements were collected at the leader for Option 1 and at the last member to join the group for Option 3.

Joining a group by processing the Welcome message exhibits a linear scaling pattern with group size across all measured metrics. This follows logically because our Welcome messages (in both Options 1 and 3) carry group state information, including the public ratchet tree and individualized encrypted group secrets, which grow directly in proportion to the group size. This finding is consistent with [11].

The overhead for group key updates via applying a commit message shows mixed trends but generally favors Option 3 in terms of bandwidth (message sizes) and processing time. Memory usage favored Option 1 due to the sparseness of intermediate nodes. We observed step-wise increases in both processing time and memory usage when group sizes grew to the next higher power of two, consistent with [11]. Their analysis, corroborated by [22], suggests this jump is due to processing the MLS ratchet tree as it grows an additional layer deeper, and we concur. The largest deltas in the sawtooth pattern of Option 3’s message sizes also occur when the group size increments by one from a perfect power of two. The sawtooth pattern occurs due to taking measurements at the last member to join the group whose location and depth in the tree changes as group size increases. At group sizes equal to powers of two (i.e. ‘a perfect tree’ described in [22]), the message sizes fit a logarithmic regression model with a high fidelity fit of $R^2 = 0.9998$. Otherwise, a harmonically repeating sawtooth pattern that trends logarithmically is observed for Option 3 message sizes. On the other hand, Option 1 update (commit) message sizes increased linearly, as expected, due to having the worst-case ratchet tree configuration.

Overall, our results confirm that Option 3 outperforms Option 1 and that our implementation generally scales linearly. We observed that high-fidelity logarithmic scaling occurs only under ideal group sizes within Option 3’s configuration for bandwidth (message size) overhead, as supported by analysis from [22]. Otherwise, a sawtooth scaling pattern with a logarithmic tendency occurs. In practical use cases, dynamic group membership can result in suboptimal ratchet tree configurations, so we do not expect this pattern to hold.

VI. FUTURE WORK

This work focused on the viability of integrating MLS with BPsec using modest design choices and assumptions; future work should push these boundaries. Scaled testing on the magnitude of thousands of group members using devices

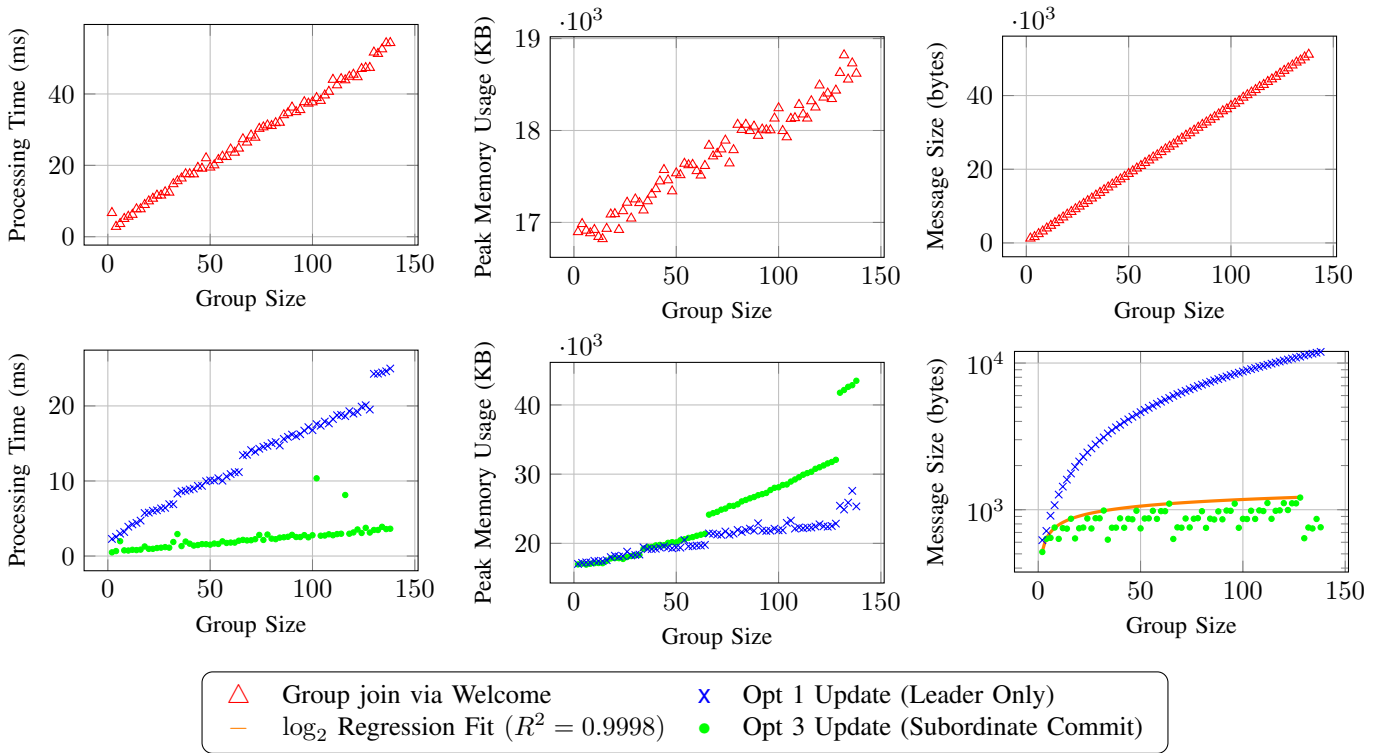


Fig. 4: Group size effect on memory, processing time, and message size for group updates and joins (via Welcome messages). Note the use of logarithmic axis (bottom right) to compare the order of magnitude difference between message sizes for Option 1 vs. 3 updates. Logarithmic regression fit based on data from group sizes of powers of two.

more representative of fielded systems may be warranted, especially for settings where a leader-subordinate paradigm is undesirable. Alternative DS designs (see Appendix A), such as an eventually consistent design under a distributed network, should also be explored to make BPsec-MLS more useful in wider settings. Furthermore, due to the longevity of space missions, strategies to employ quantum-resistant BPsec-MLS (e.g., using NIST-recommended ML-KEM and ML-DSA) in efficient manners should be explored now in preparation for the emergence of cryptographically relevant quantum computers during the lifetime of space systems being fielded today.

VII. CONCLUSION

The integration of MLS with BPsec marks a significant advancement in securing space communications that utilize the DTN stack. By leveraging MLS, BPsec gains efficient group key management and security features such as forward secrecy and post-compromise security, which were previously unattainable through traditional methods. Our implementation of BPsec-MLS, tested in two contrasting configurations under a DTN scenario, reveals both practical limitations and actionable solutions, paving ways to achieve more secure and efficient space network architectures. While linear scaling remains the norm except under ideal conditions that show logarithmic tendencies, our findings highlight the potential for future optimizations and broader adoption.

REFERENCES

- [1] J. Schier, “Interoperability in the Lunar Sphere through LunaNet and International Cooperation,” June 2024.
- [2] Starlink, “Starlink 2024 progress report,” tech. rep., Starlink, 2024.
- [3] E. S. Agency, “Report on the Space Economy 2024,” tech. rep., Dec. 2024.
- [4] D. J. Israel, K. D. Mauldin, C. J. Roberts, J. W. Mitchell, A. A. Pulkkinen, L. V. D. Cooper, M. A. Johnson, S. D. Christe, and C. J. Gramling, “Lunaret: a flexible and extensible lunar exploration communications and navigation infrastructure,” in *2020 IEEE Aerospace Conference*, pp. 1–14, 2020.
- [5] J. Smailes, S. Köhler, S. Birnbach, M. Strohmeier, and I. Martinovic, “KeySpace: Public Key Infrastructure Considerations in Interplanetary Networks,” Nov. 2024. arXiv:2408.10963 [cs].
- [6] M. Parisi, T. Panontin, S.-C. Wu, K. Mctigue, and A. Vera, “Effects of Communication Delay on Human Spaceflight Missions,” 2023.
- [7] L. Torgerson, S. C. Burleigh, H. Weiss, A. J. Hooke, K. Fall, D. V. G. Cerf, K. Scott, and R. C. Durst, “Delay-Tolerant Networking Architecture,” RFC 4838, Apr. 2007.
- [8] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon, “The Messaging Layer Security (MLS) Protocol,” RFC 9420, July 2023.
- [9] M. Anastos, B. Auerbach, M. A. Baig, M. C. Noval, M. Kwan, G. Pascual-Perez, and K. Pietrzak, “The cost of maintaining keys in dynamic groups with applications to multicast encryption and group messaging,” in *Theory of Cryptography: 22nd International Conference, TCC 2024, Milan, Italy, December 2–6, 2024, Proceedings, Part I*, (Berlin, Heidelberg), p. 413–443, Springer-Verlag, 2024.
- [10] J. Alwen, D. Hartmann, E. Kiltz, and M. Mularczyk, “Server-aided continuous group key agreement,” in *ACM CCS 2022* (H. Yin, A. Stavrou, C. Cremers, and E. Shi, eds.), pp. 69–82, ACM Press, Nov. 2022.
- [11] D. Soler, C. Dafonte, M. Fernández-Veiga, A. F. Vilas, and F. J. Nóvoa, “Experimental analysis of efficiency of the messaging layer security for multiple delivery services,” *arXiv preprint arXiv:2502.18303*, 2025.

- [12] B. Dowling, B. Hale, X. Tian, and B. Wimalasiri, “Key establishment in the space environment,” 2025.
- [13] J. Alwen, S. Coretti, and Y. Dodis, “The double ratchet: Security notions, proofs, and modularization for the Signal protocol,” in *EUROCRYPT 2019, Part I* (Y. Ishai and V. Rijmen, eds.), vol. 11476 of *LNCS*, pp. 129–158, Springer, Cham, May 2019.
- [14] J. Alwen, S. Coretti, Y. Dodis, and Y. Tselekounis, “Security analysis and improvements for the ietf mls standard for group messaging,” in *Advances in Cryptology – CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part I*, (Berlin, Heidelberg), p. 248–277, Springer-Verlag, 2020.
- [15] T. Wallez, J. Protzenko, and K. Bhargavan, “TreeKEM: A Modular Machine-Checked Symbolic Security Analysis of Group Key Agreement in Messaging Layer Security,” 2025. Publication info: Preprint.
- [16] C. Hale and B. Kohbrok, “The complexities of healing in secure group messaging: Why cross-group effects matter,” pp. 1847–1864, 2021.
- [17] C. Brzuska, E. Cornelissen, and K. Kohbrok, “Security Analysis of the MLS Key Distribution,”
- [18] N. Sullivan and S. Turner, “Messaging layer security: Secure and usable end-to-end encryption,” IETF Blog, 2024.
- [19] B. Beurdouche, E. Rescorla, E. Omara, S. Inguva, and A. Duric, “The Messaging Layer Security (MLS) Architecture,” Internet-Draft draft-ietf-mls-architecture-15, Internet Engineering Task Force, Aug. 2024. Work in Progress.
- [20] A. Bienstock, Y. Dodis, S. Garg, G. Grogan, M. Hajiabadi, and P. Rösler, “On the worst-case inefficiency of cgka,” in *Theory of Cryptography: 20th International Conference, TCC 2022, Chicago, IL, USA, November 7–10, 2022, Proceedings, Part II*, (Berlin, Heidelberg), p. 213–243, Springer-Verlag, 2022.
- [21] M. Anastos, B. Auerbach, M. A. Baig, M. C. Noval, M. Kwan, G. Pascual-Perez, and K. Pietrzak, “The Cost of Maintaining Keys in Dynamic Groups with Applications to Multicast Encryption and Group Messaging,” in *Theory of Cryptography: 22nd International Conference, TCC 2024, Milan, Italy, December 2–6, 2024, Proceedings, Part I*, (Berlin, Heidelberg), pp. 413–443, Springer-Verlag, Dec. 2024.
- [22] C. Chevalier, G. Lebrun, A. Martinelli, and J. Plüt, “The art of bonsai: How well-shaped trees improve the communication cost of MLS,” Cryptology ePrint Archive, Paper 2024/746, 2024.
- [23] S. Burleigh, K. Fall, and E. J. Birrane, “Bundle Protocol Version 7,” RFC 9171, Jan. 2022.
- [24] E. J. Birrane and K. McKeever, “Bundle Protocol Security (BPsec),” RFC 9172, Jan. 2022.
- [25] B. Dowling, B. Hale, X. Tian, and B. Wimalasiri, “Cryptography is rocket science,” *cic*, vol. 1, Issue 4, 2025.
- [26] E. J. Birrane, A. White, and S. Heiner, “Default Security Contexts for Bundle Protocol Security (BPsec),” RFC 9173, Jan. 2022.
- [27] M. Sommer, A. Sterz, M. Vogelbacher, H. Bellafkir, and B. Freisleben, “Quic: A quick convergence layer for disruption-tolerant networks,” in *Proceedings of the Int’l ACM Conference on Modeling Analysis and Simulation of Wireless and Mobile Systems, MSWiM ’23*, (New York, NY, USA), p. 37–46, Association for Computing Machinery, 2023.
- [28] B. Dowling, B. Hale, K. Kohbrok, R. Robert, X. Tian, and B. Wimalasiri, “Securing QUIC with MLS,” Internet-Draft draft-tian-quic-quicmls-00, Internet Engineering Task Force, July 2025. Work in Progress.
- [29] A. Hülsing, T. Lange, and F. Weber, “Key-Update Mechanism for SDLSP,” May 2024.
- [30] S. A. Menesidou and V. Katos, “Authenticated Key Exchange (AKE) in Delay Tolerant Networks,” in *Information Security and Privacy Research* (D. Gritzalis, S. Furnell, and M. Theoharidou, eds.), vol. 376, pp. 49–60, Berlin, Heidelberg: Springer Berlin Heidelberg, 2012. Series Title: IFIP Advances in Information and Communication Technology.
- [31] F. Fuchs, F. Walter, and F. Tschorsch, “BERMUDA: A BPsec-compatible key management scheme for DTNs,” Cryptology ePrint Archive, Paper 2025/806, 2025.
- [32] W. L. Van Besien, “Dynamic, non-interactive key management for the bundle protocol,” in *Proceedings of the 5th ACM Workshop on Challenged Networks, CHANTS ’10*, (New York, NY, USA), p. 75–78, Association for Computing Machinery, 2010.
- [33] K. Cohn-Gordon, C. Cremers, and L. Garratt, “On Post-compromise Security,” in *2016 IEEE 29th Computer Security Foundations Symposium (CSF)*, pp. 164–178, IEEE, 6 2016.
- [34] J. Alwen, D. Jost, and M. Mularczyk, “On the insider security of MLS,” in *CRYPTO 2022, Part II* (Y. Dodis and T. Shrimpton, eds.), vol. 13508 of *LNCS*, pp. 34–68, Springer, Cham, Aug. 2022.
- [35] J. H. Saltzer, D. P. Reed, and D. D. Clark, “End-to-end arguments in system design,” *ACM Trans. Comput. Syst.*, vol. 2, pp. 277–288, Nov. 1984.
- [36] B. Sipos, “Bundle Protocol Version 7 Administrative Record Types Registry,” RFC 9713, Jan. 2025.
- [37] S. C. Burleigh, S. Farrell, and M. Ramadas, “Licklider Transmission Protocol - Specification,” RFC 5326, Sept. 2008.
- [38] S. C. Burleigh, D. Horres, K. Viswanathan, M. Benson, and F. Templin, “Architecture for Delay-Tolerant Key Administration,” Internet-Draft draft-burleigh-dtnwg-dtka-02, Internet Engineering Task Force, Aug. 2018. Work in Progress.
- [39] Consultative Committee for Space Data Systems, “Rationale, Scenarios, and Requirements for DTN in Space,” vol. CCSDS 734.0-G-1, no. 1, pp. 5–7, 2010.
- [40] NASA Jet Propulsion Laboratory, “Interplanetary Overlay Network (ION-DTN),” GitHub, 2025. [Source code; Accessed: 12-Sept-2025].
- [41] K. McKeever, “RFC 9172: Bundle Protocol Security (BPsec),” 2022.
- [42] Cisco Systems, “MLSp: Implementation of Messaging Layer Security,” <https://github.com/cisco/mlspp>, 2025. C++ implementation of the Messaging Layer Security protocol.

APPENDIX A

DELIVERY SERVICE (DS) CONSIDERATIONS

The MLS architecture in [19] discusses two types of delivery services for MLS: *strongly consistent* and *eventually consistent*. A strongly consistent DS is found in MLS implementations in traditional internet settings that utilize reliable delivery and ordering guarantees provided by TCP or a centralized server. An eventually consistent DS, which is more appropriate for a distributed peer-to-peer setting utilizes clients for reconciliation of conflicts.¹

Several approaches for a DS are possible for the DTN setting. The most straight-forward is the strongly consistent DS which would use convergence layer adapters like TCP Convergence Layer (TCPCL) or Licklider Transmission Protocol (LTP) for reliable in-order delivery of MLS messages. An eventually consistent DS is more topologically flexible but is more complex, requiring considerably more storage, local processing, and bandwidth to store previous group states and either attach or solicit missing/unconfirmed commit messages. Soler et al. in [11] explore using the MQTT based GossipSub protocol and distributed hash tables to implement such an eventually consistent delivery service and found efficiency losses when compared to a hub-spoke style strongly consistent DS. As a proof of concept, our DS simply relies on LTP and we appoint a fixed group leader through which group control messages are passed to and committed.

¹More discussion on how the CAP (Consistency, Availability, Partition Tolerance) Theorem is applied in designing a strongly consistent vs. eventually consistent DS can be found in [19]